

Performance Testing

Date	30 June 2026
Team ID	XXXXXX
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Web Browser (Manual Testing)
Type of Testing	Functional & Performance Testing
Target Module / API	Flood Prediction Module (/predict)
Test Environment	Local System (Flask Development Server)
Test Date	30 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home Page Loading	1	30 sec	Page loads successfully
2	Flood Prediction	1	60 sec	Prediction generated correctly
3	Invalid Input Testing	1	30 sec	Input validation works
4	Multiple Predictions	1	120 sec	Application remains responsive

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.8 sec	Pass	Fast
2	Response Time (Max)	< 5 seconds	1.5 sec	Pass	Acceptable
3	Throughput (Req/sec)	-	15 req/sec	Pass	Stable
4	Error Rate	< 1%	0%	Pass	No errors
5	CPU Utilization	< 80%	35%	Pass	Normal
6	Memory Utilization	< 80%	42%	Pass	Stable

Step 4: Observations & Analysis

Key Findings:

The application responded quickly to prediction requests and maintained stable performance during repeated testing. No significant delays or failures were observed.

Bottlenecks Identified:

No major performance bottlenecks were identified during local testing.

Optimization Steps Taken:

Optimized the XGBoost model loading, organized the Flask application into modular components, and minimized redundant processing.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

